Machine Learning and Deep Learning
Lecture-9

By: Somnath Mazumdar
Assistant Professor
sma.digi@cbs.dk

Spring 2026

Overview

- Intro to Artificial Neural Networks
- Perceptron
  - TLU
  - MLP
- Dropout
- Batch norm

Artificial
Neural Networks

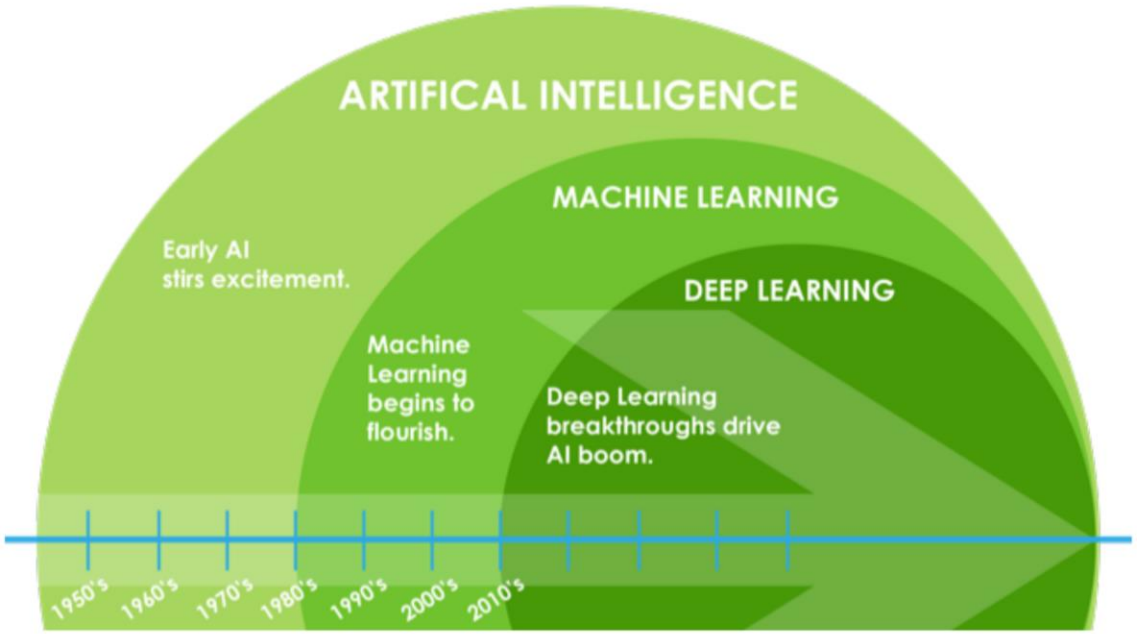
ML and Deep Learning History

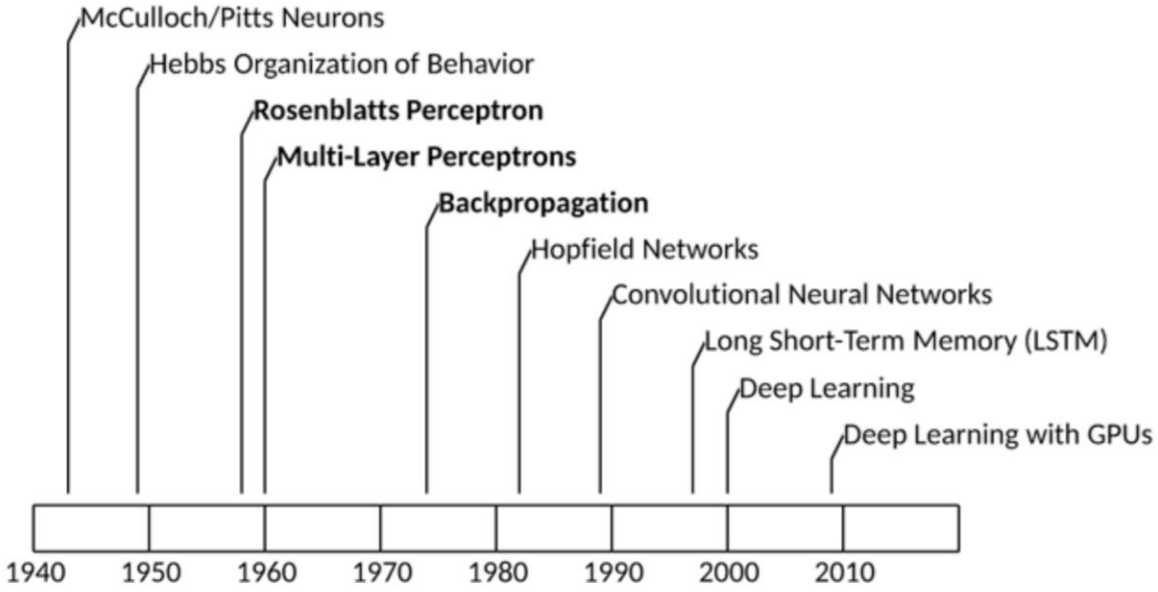
- 1950 and 1960s: Initial excitement.
  - Perceptron: linear classifier and stochastic gradient (roughly).
  - Drop in popularity: Realized limitations of linear models.

- 1970 and 1980s: Connectionism (brain-inspired ML)
  - Want "connected networks of simple units".
  - Adding hidden layers z_i increases expressive power.
  - Success in optical character recognition.

- 1990s and early-2000s: drop in popularity.
  - Rise in popularity of logistic regression and SVMs with regularization and kernels.

ML Evolution





Overview of ML

- Components: Data, Model, Objective Function, Optimization Algorithm.
- Data: Input and output data
  - Common way of describing an input dataset is with a design matrix.
  - Issues: underfitting, overfitting, training error and test error.

- Model: To generate output.
  - Parametric models (linear regression)
  - Non-parametric models.

- Objective function: Real valued function that provides a measure of how wrong model is in terms of its ability to estimate relationship between inputs and outputs.

Overview of ML

- Objective function: real values function that provides a measure of how wrong the model is in terms of its ability to estimate the relationship between inputs and outputs.
    - Mean square error function, cross-entropy function (classification problems), Maximum Likelihood estimation.

- Optimization Algorithm: Minimizes or maximizes function f (x) by altering x.
    - Gradient descent: reduce f(x) by moving x in opposite sign of derivative.

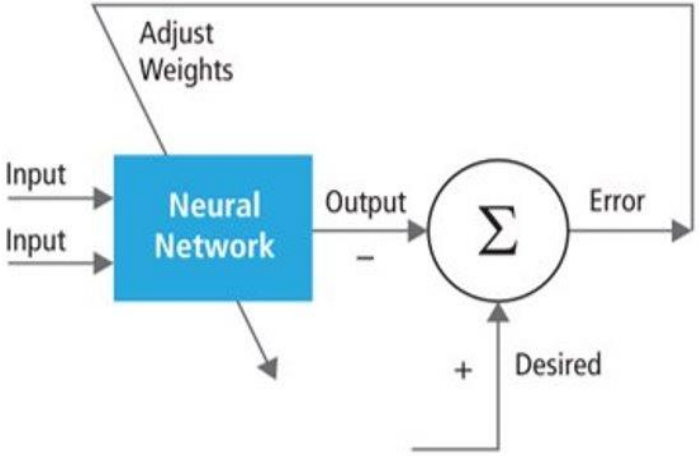
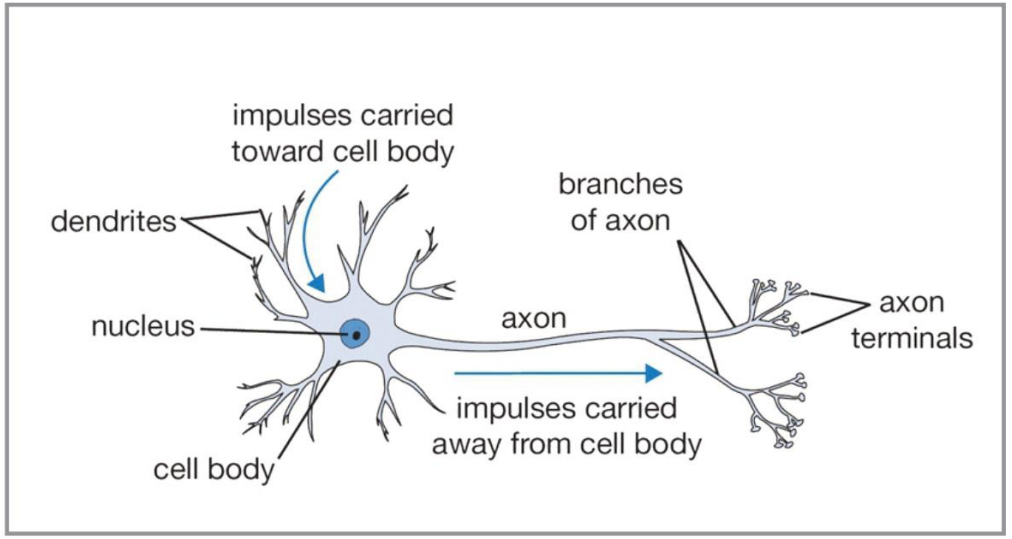
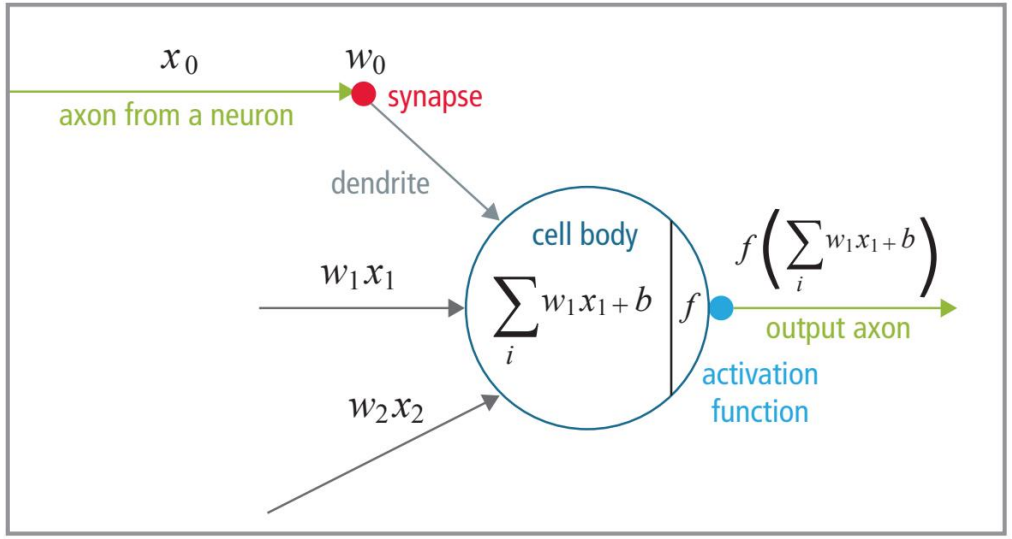


Image Src: Using Convolutional Neural Networks for Image Recognition By Samer Hijazi, Rishi Kumar, and Chris Rowen, IP Group, Cadence

From Biological to Artificial Neurons

- First introduced* in 1943 by Warren McCulloch and Walter Pitts.
- First ANN: To show of how biological neurons might work together in animal brains to perform complex computations using propositional logic.

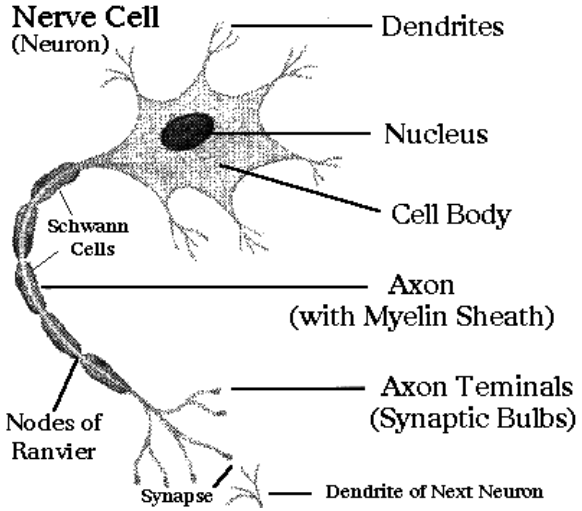


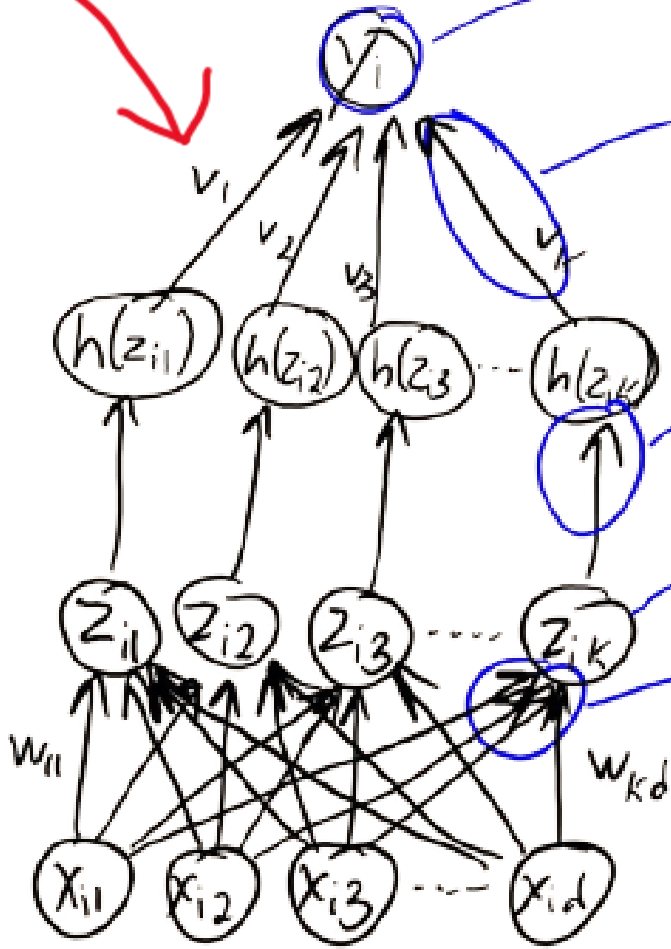


- “Dendrites” takes input signal.
- “Axon” sends an output signal.

*Warren S. McCulloch and Walter Pitts, “A Logical Calculus of the Ideas Immanent in Nervous Activity,” The Bulletin of Mathematical Biology 5, no. 4 (1943): 115–113.
Image Src: Using Convolutional Neural Networks for Image Recognition By Samer Hijazi, Rishi Kumar, and Chris Rowen, IP Group, Cadence

Why “Neural Network”?





Predictions based on aggregation $v^T h(Wx_i)$ at y_i "neuron"

Synapse between z_{ik} and y_i "neuron"

binary signal $h(w_c^T x_i)$ sent along "axon"

Neuron aggregates signals: $w_c^T x_i$

"dendrites" for z_{ik} "neuron" are reciving x_{ij} values

CPSC 340: Machine Learning and Data Mining: http://www.stat.yale.edu/Courses/ 1997-98/101/stat101.htm

Notations

Symbol	Meaning
x_i	Input vector
$W^{(l)}$	Weight matrix at layer l
$z_i^{(l)}$	Linear pre-activation
$h(\cdot)$	Nonlinear activation
$a_i^{(l)}$	Activated output of layer l
v, w	Output layer weights
$\hat{y}_i$	Model prediction

x_{ij}

Feature j of example i

$z_{ij}^{(l)}$

Pre-activation of neuron j in layer l for example i

$h(z_{ij}^{(l)})$

Post-activation (after applying nonlinearity)

Arrows

Represent learned weights.

"Latent features"

Each hidden layer learns intermediate representations (feature transformations).

1. Linear Model (No Hidden Layer)

$$\hat{y}_i = w^\top x_i$$

Notation

- i : index of the training example
- $x_i \in \mathbb{R}^d$: input feature vector
- $w \in \mathbb{R}^d$: weight vector
- $w^\top x_i$: dot product (linear combination)
- $\hat{y}_i$: predicted output

3. Two Hidden Layers

$$\hat{y}_i = v^\top h\left(W^{(2)} h\left(W^{(1)} x_i\right)\right)$$

Layer-wise notation

- $W^{(1)}$: weights for layer 1
- $W^{(2)}$: weights for layer 2
- Superscript (l) = layer index

Intermediate variables:

$$z_i^{(1)} = W^{(1)} x_i$$

$$a_i^{(1)} = h(z_i^{(1)})$$

$$z_i^{(2)} = W^{(2)} a_i^{(1)}$$

$$a_i^{(2)} = h(z_i^{(2)})$$

Final output:

$$\hat{y}_i = v^\top a_i^{(2)}$$

2. One Hidden Layer Network

$$\hat{y}_i = v^\top h(W x_i)$$

New notation

- $W \in \mathbb{R}^{k \times d}$: weight matrix from input → hidden layer
- $z_i = W x_i \in \mathbb{R}^k$: hidden pre-activation vector
- $h(\cdot)$: element-wise nonlinearity (ReLU, sigmoid, tanh, etc.)
- $h(z_i) \in \mathbb{R}^k$: hidden activations
- $v \in \mathbb{R}^k$: weights from hidden layer → output
- $v^\top h(W x_i)$: final linear combination

Interpretation

1. Linear transform: $z_i = W x_i$
2. Nonlinear transform: $a_i = h(z_i)$
3. Output layer: $\hat{y}_i = v^\top a_i$

Deep Learning

Linear model:

$$\hat{y}_i = w^T x_i$$

Neural network with 1 hidden layer:

$$\hat{y}_i = v^T h(\underbrace{Wx_i}_{z_i})$$

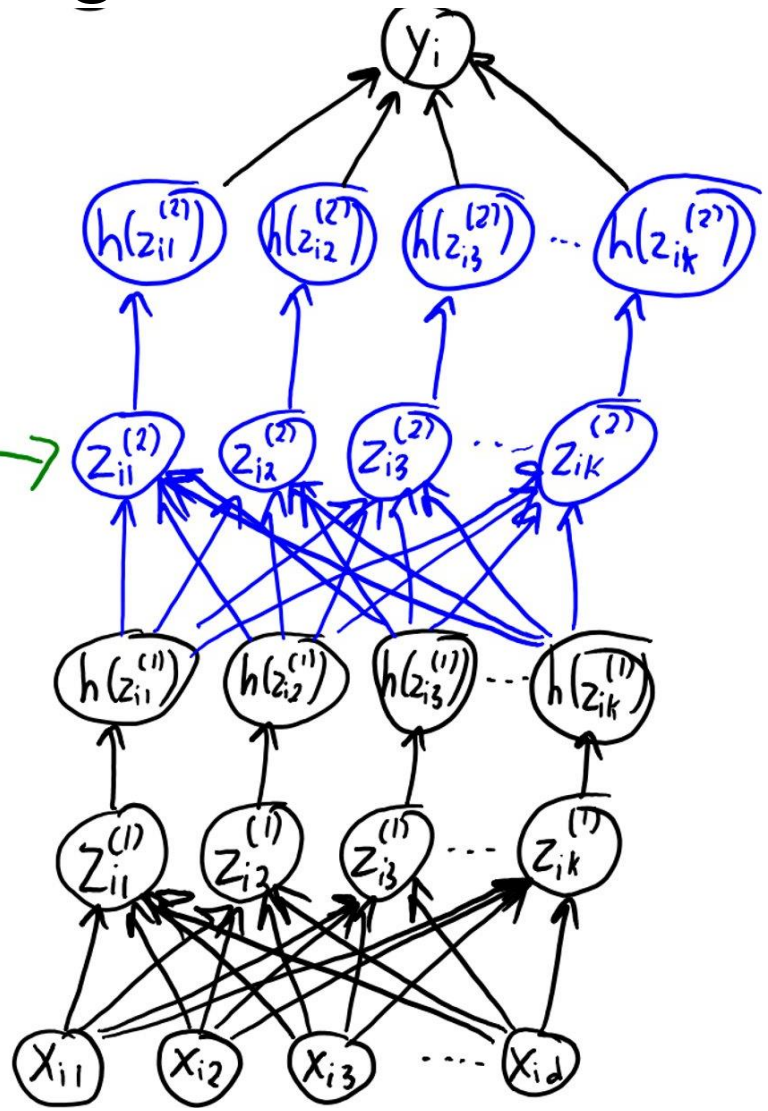
Neural network with 2 hidden layers:

$$\hat{y}_i = v^T h(\underbrace{W^{(2)} h(\underbrace{W^{(1)} x_i}_{z_i^{(1)}})}_{z_i^{(2)}})$$

Neural network with 3 hidden layers

$$\hat{y}_i = v^T h(\underbrace{W^{(3)} h(\underbrace{W^{(2)} h(\underbrace{W^{(1)} x_i}_{z_i^{(1)}})}_{z_i^{(2)}})}_{z_i^{(3)}})$$

Deep learning:



Second "layer" of latent features

You can add more "layers" to go "deeper"

- For 'm' layers, we could use: $\hat{y}_i = w^T \left(\prod_{\ell=1}^{m} h(W^{(\ell)} x_i) \right)$
- NOTE: Non-linearity: Transform z_i by non-linear function 'h': $z_i = Wx_i$ ==> $y_i = v^T h(z_i)$

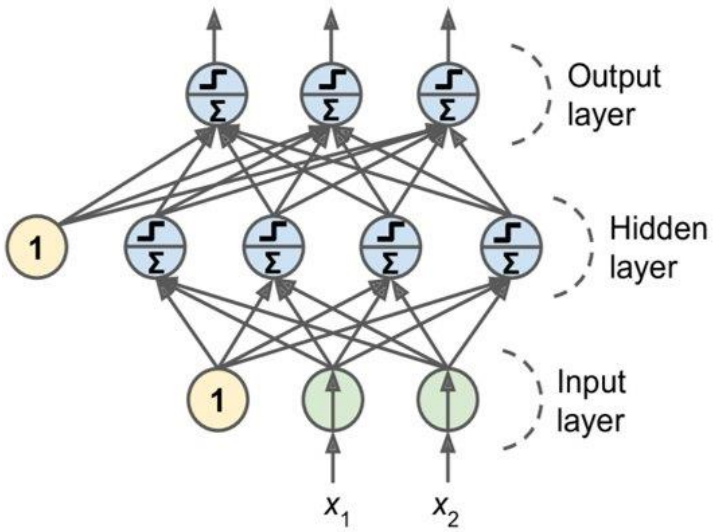
Bias

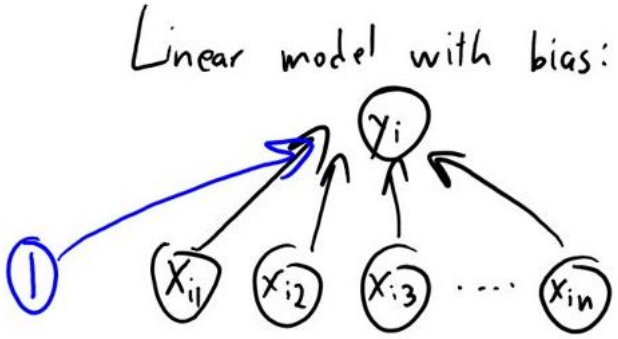
- Generalization error consists of three different errors: Bias, Variance, Irreducible error (due to noisiness of data).

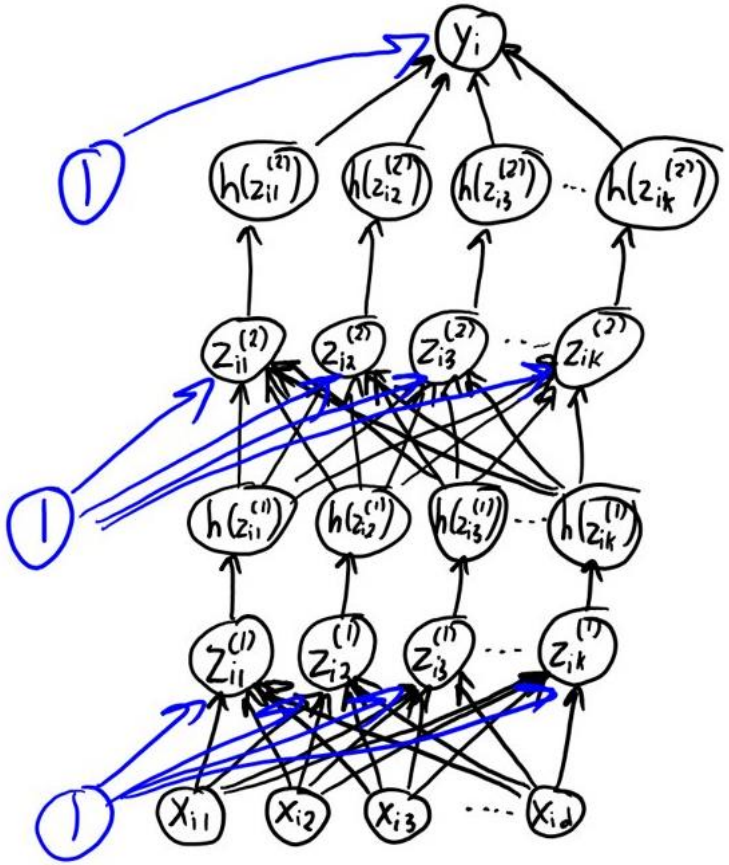
- Bias happen due to wrong assumptions.
- High-bias model is most likely to underfit the training data.
- Example: Assuming data is linear when it is quadratic.

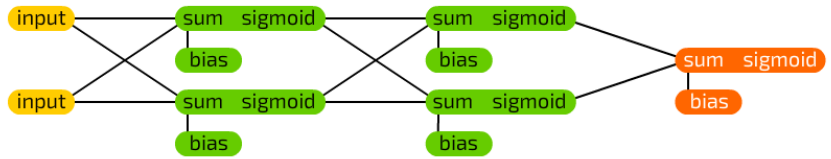
- Note: If you trains a linear SVM model using the LinearSVC class (of Sci-kit).
  - LinearSVC class regularizes bias term.

Adding Bias

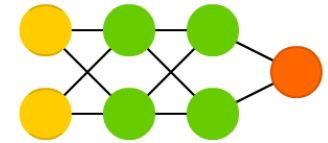

Output layer
Hidden layer
Input layer
1
1
x_1
x_2

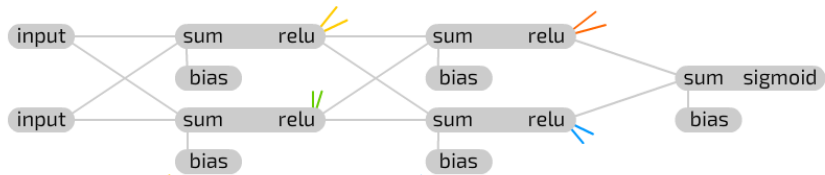

Linear model with bias:
y_i
1
x_{i1}
x_{i2}
x_{i3}
x_{in}


y_i
1
$h(z_{i1}^{(2)})$
$h(z_{i2}^{(2)})$
$h(z_{i3}^{(2)})$
$h(z_{ik}^{(2)})$
$z_{i1}^{(2)}$
$z_{i2}^{(2)}$
$z_{i3}^{(2)}$
$z_{ik}^{(2)}$
1
$h(z_{i1}^{(1)})$
$h(z_{i2}^{(1)})$
$h(z_{i3}^{(1)})$
$h(z_{ik}^{(1)})$
$z_{i1}^{(1)}$
$z_{i2}^{(1)}$
$z_{i3}^{(1)}$
$z_{ik}^{(1)}$
1
x_{i1}
x_{i2}
x_{i3}
x_{id}

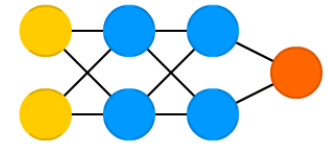


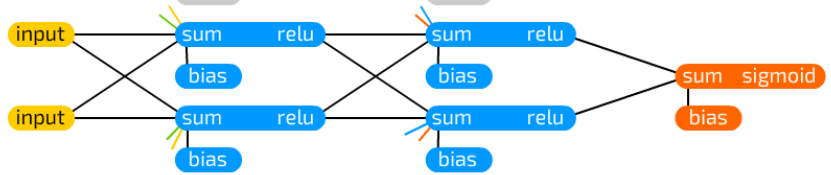
Deep Feed Forward Example



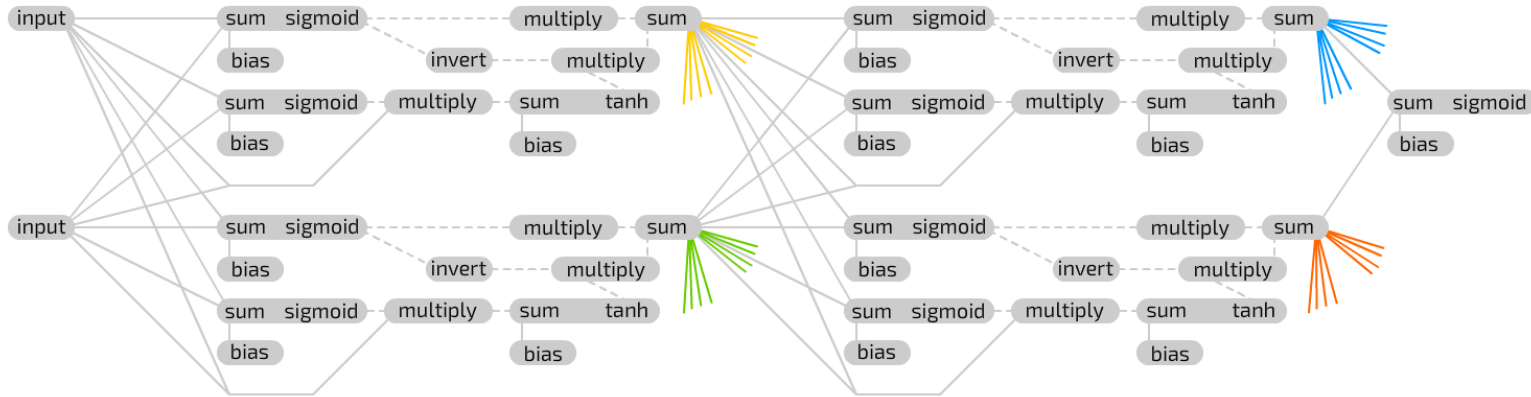


Deep Recurrent Example
(previous iteration)

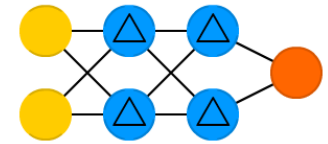


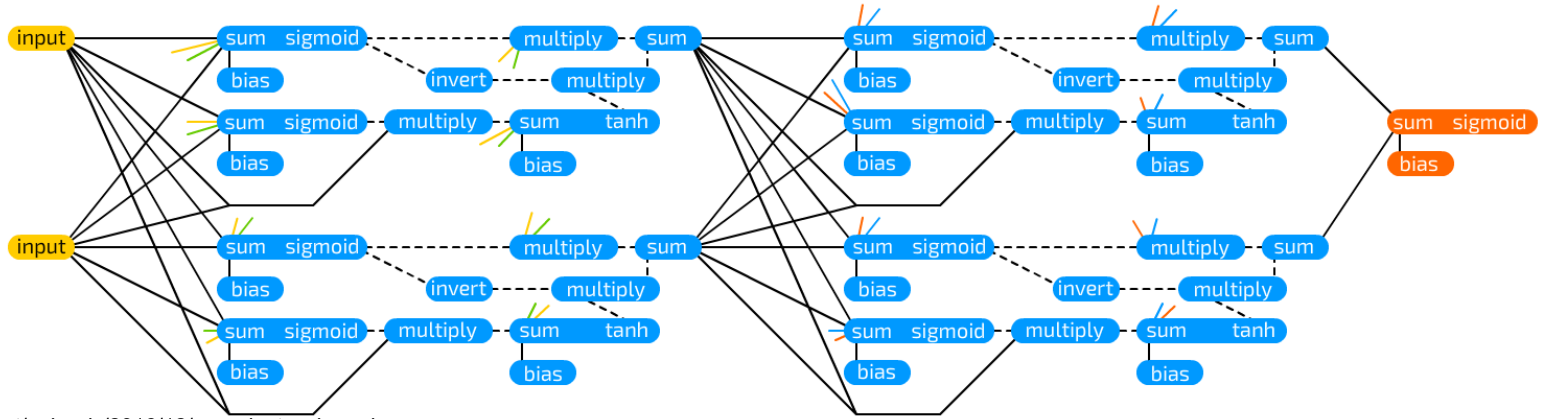


Deep Recurrent Example



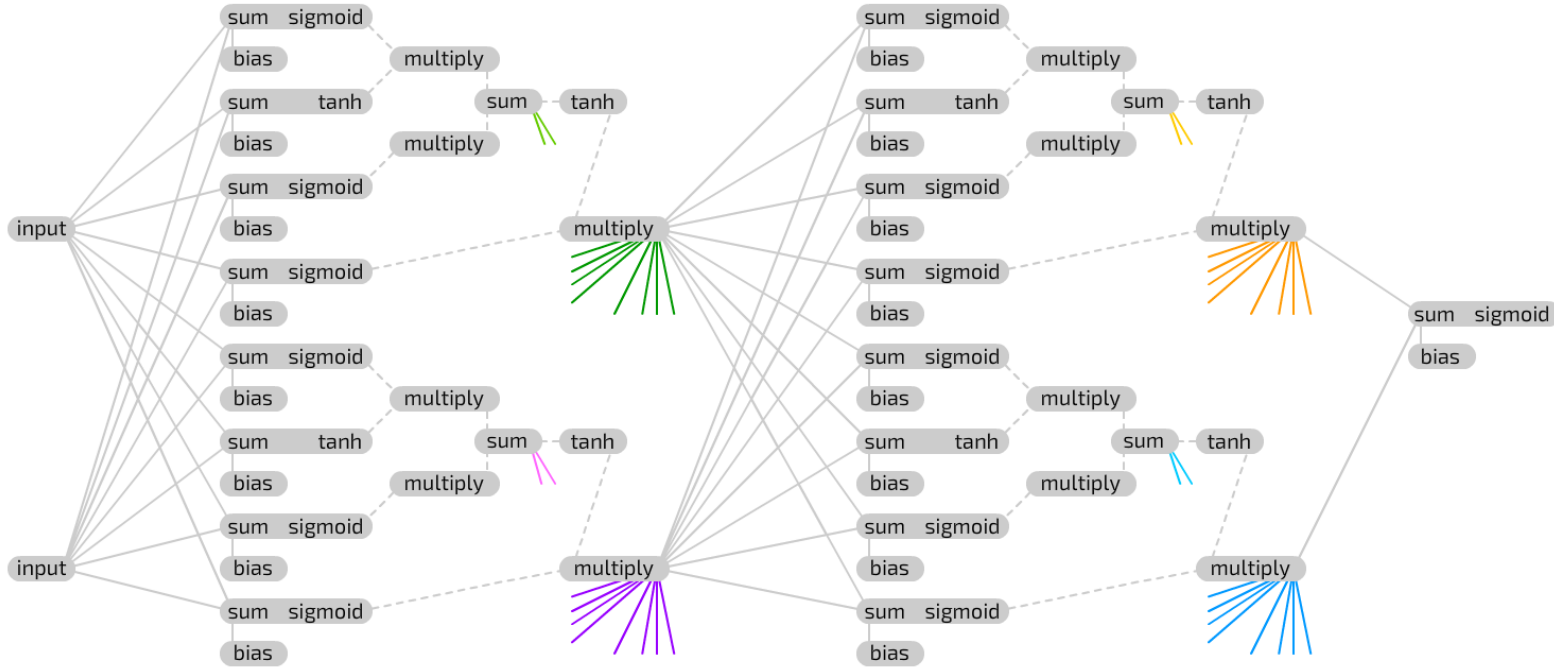
Deep GRU Example
(previous iteration)



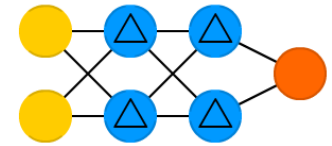


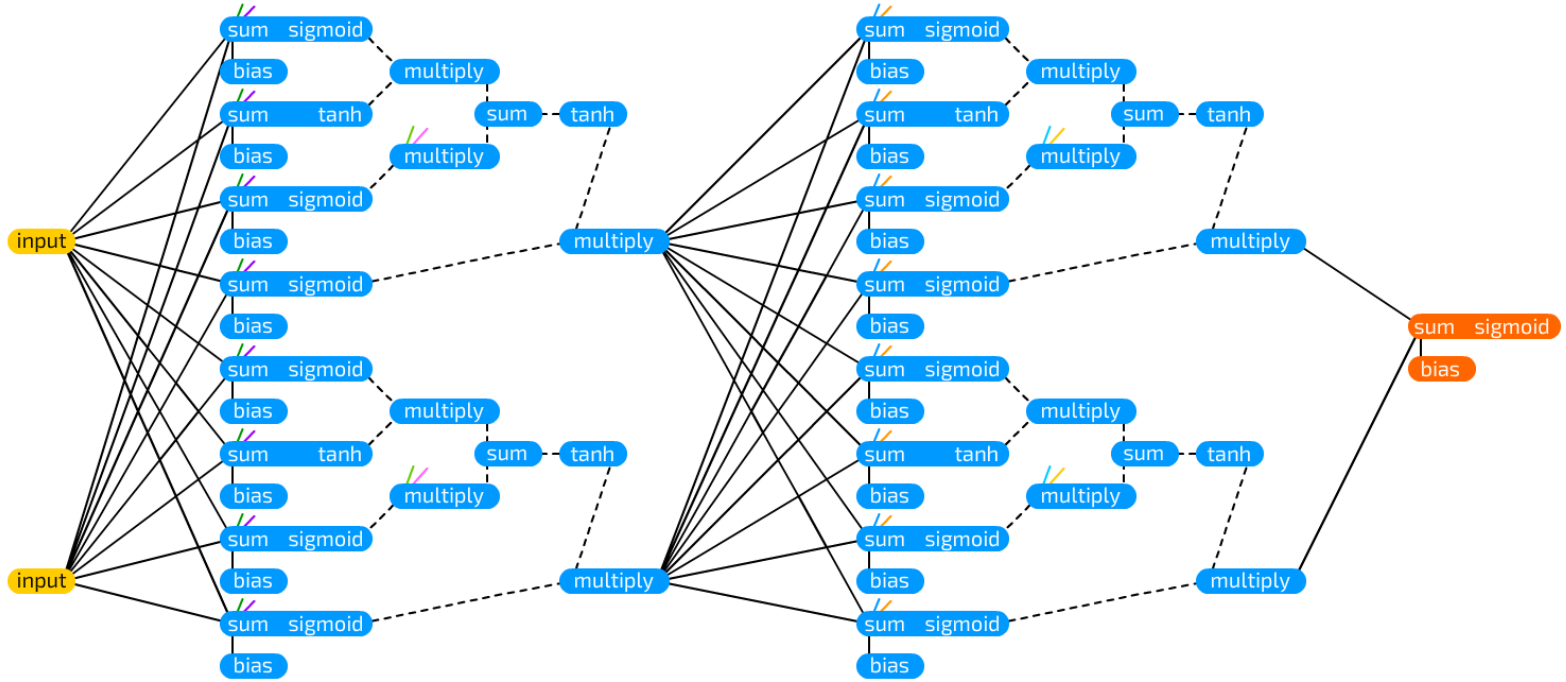
Deep GRU Example

https://www.asimovinstitute.org/wp-content/uploads/2016/12/neuralnetworkgraphs.png



Deep LSTM Example
(previous iteration)





Deep LSTM Example

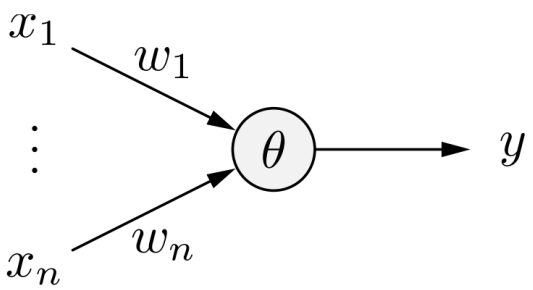
https://www.asimovinstitute.org/wp-content/uploads/2016/12/neuralnetworkgraphs.png

Threshold Logic Units, Perceptron, & Multilayer Perceptron

Threshold Logic Units (TLUs)

- TLU is a processing unit for numbers with n inputs $x_1,\ldots,x_n$ and one output y.
- Unit has a threshold θ and each input x_i is associated with a weight w_i.
- Threshold logic unit computes the function.

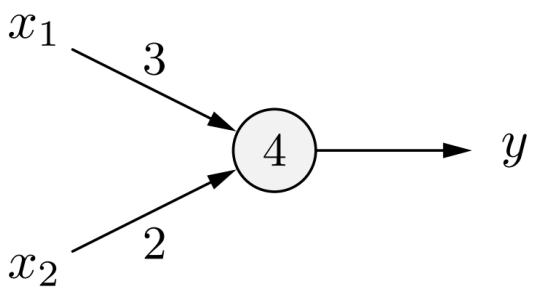
$$y = \begin{cases} 1, & \text{if} \quad \sum_{i=1}^{n} w_i x_i \geq \theta, \\ 0, & \text{otherwise.} \end{cases}$$



- TLUs mimic thresholding behaviour of biological neurons in a (very) simple fashion.

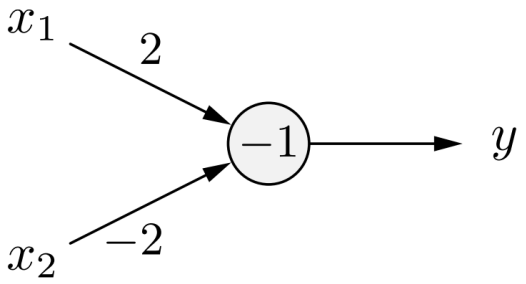
Example

- Threshold logic unit for the conjunction x1∧x2.



x_1	x_2	$3x_1 + 2x_2$	y
0	0	0	0
1	0	3	0
0	1	2	0
1	1	5	1

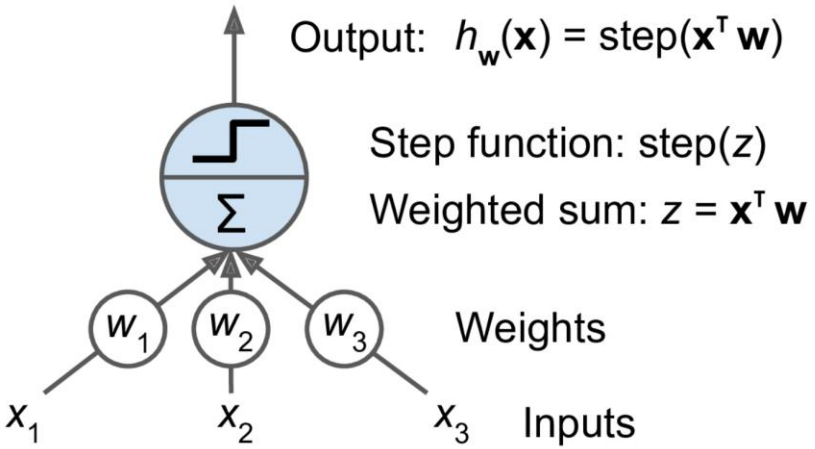
- Threshold logic unit for the implication x2→x1.



x_1	x_2	$2x_1 - 2x_2$	y
0	0	0	1
1	0	2	1
0	1	-2	0
1	1	0	1

Perceptron

- Simplest ANN architectures.
- Invented in 1957 by Frank Rosenblatt.
- Based on a slightly different artificial neuron called a threshold logic unit (TLU), or linear threshold unit (LTU).



- Inputs and output are numbers, and each input connection is associated with a weight.
- TLU computes a weighted sum of its inputs, then applies a step function to that sum and outputs result.

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurélien Géron, 2019.

Perceptron

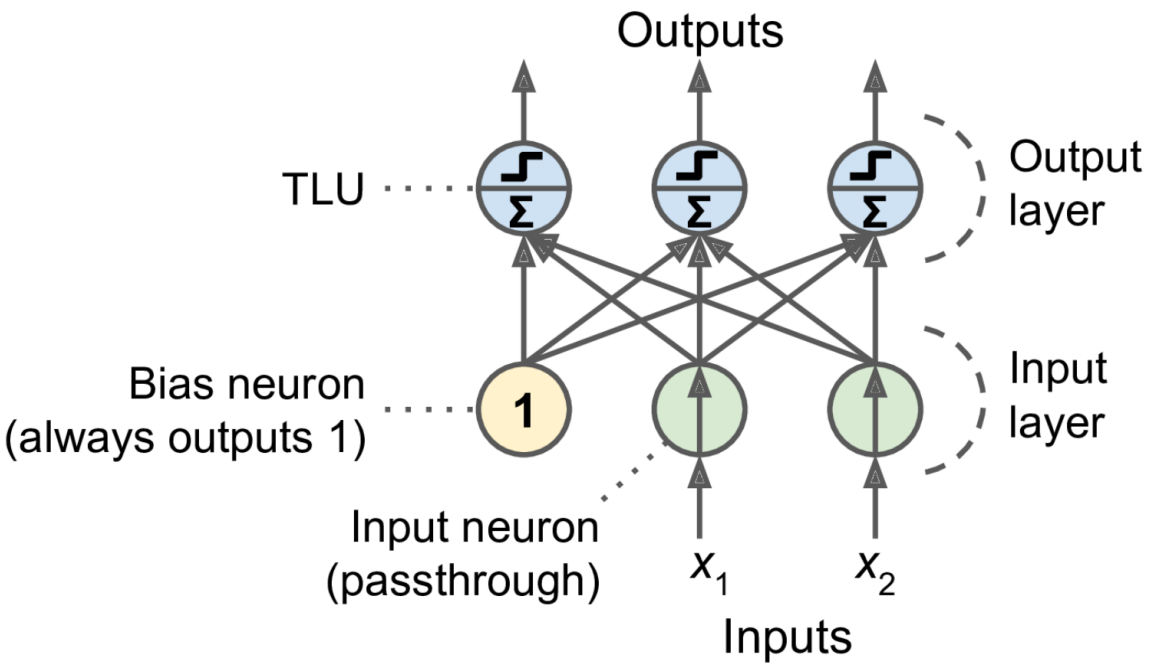
- Single TLU can be used for simple linear binary classification.
- Perceptron is composed of a single layer of TLUs.
- Each TLU connected to all inputs.
  - TLU computes a linear combination of inputs.
  - If result exceeds a threshold, it outputs positive class.

  - Fully connected /dense layer: All neurons in a layer connected to every neuron in previous layer.

- An extra bias feature is generally added ($x_0 = 1$):
  - Typically represented using a special type of neuron called a bias neuron.
  - Outputs 1 always.

Example

- Perceptron with two inputs and three outputs
- This Perceptron can classify instances simultaneously into three different binary classes, which makes it a multi-output classifier.



Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurélien Géron, 2019.

Perceptron

- How is a Perceptron trained?
  - Perceptron training algorithm was largely inspired by Hebb's rule (or Hebbian learning).
- Process:
  1. Perceptron is fed one training instance at a time, and for each instance it makes its predictions.
  2. For every output neuron that produced a wrong prediction, it reinforces the connection weights from the inputs that would have contributed to the correct prediction.
- Heaviside step function used in Perceptrons.
- Common step functions used in Perceptrons (assuming threshold = 0)

$$\text{heaviside}\,(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases} \qquad \text{sgn}\,(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$

signum function extracts the sign of a real number z.

- Perceptron and SGDClassifier both have similar implementation.

Perceptron

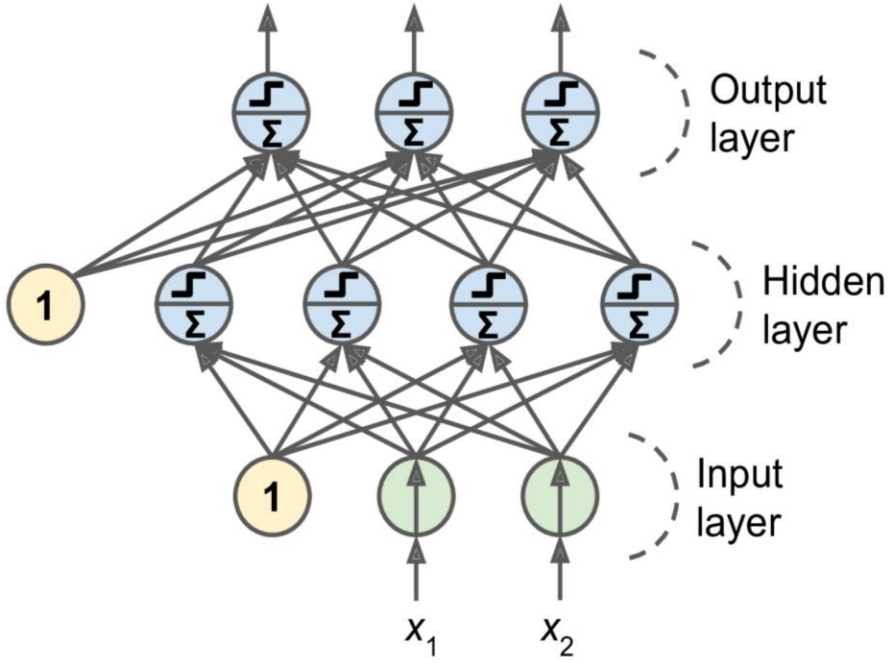
- Perceptron incapable of learning complex patterns:
  - Because decision boundary of each output neuron is linear (just like Logistic Regression classifiers).

- Logistic Regression Vs Perceptrons:
  - Perceptrons do not output a class probability.
  - Perceptrons make predictions based on a hard threshold.

- Weaknesses:
  - Incapable of solving Exclusive OR (XOR) classification problem
  - Soln: Can be solved by stacking multiple Perceptrons
    - Resulting ANN is called a Multilayer Perceptron (MLP).

Multilayer Perceptron: MLP

- MLP is composed of one input layer, one or more layers of TLUs, called hidden layers*, and a layer of TLU called output layer.
- Layers close to input layer are called lower layers
- Layers close to outputs are called upper layers.
- Every input/hidden layer includes a bias neuron and is fully connected to next layer.

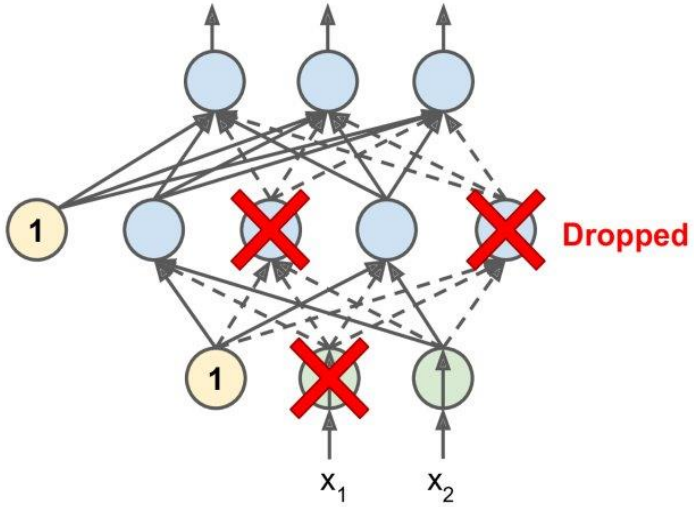


MLP with 2-inputs, one hidden layer of 4-neurons, and 3-output neurons

*When an ANN contains a deep stack of hidden layers it is called a deep neural network (DNN).

Regularization: Dropout

Dropout*



- Popular regularization techniques for deep neural networks.
- NNs get a 1–2% accuracy boost simply by adding dropout.

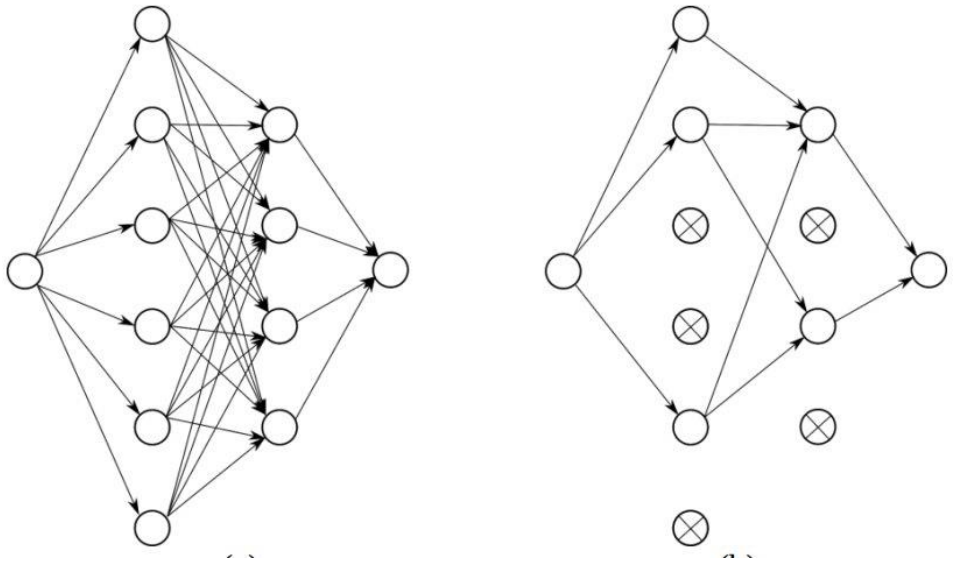
- Working: At every training step, every input neurons has a probability (p) of being temporarily "dropped out."
  - Means it will be entirely ignored during this training step, but it may be active during next step.
  - After training, neurons don't get dropped anymore.

- Hyperparameter p is called dropout rate (set between 10% and 50%):
- E.g, RNN (closer to 20-30%), CNN(closer to 40-50%).

*Hinton, Geoffrey E., et al. "Improving neural networks by preventing co-adaptation of feature detectors." *arXiv preprint arXiv:1207.0580* (2012).

Dropout

- In this example: Apply dropout only to neurons in top one-to-three layers (excluding output layer).
  - Suppose p = 50%: During testing a neuron would be connected to twice as many input neurons as it would be (on average) during training.

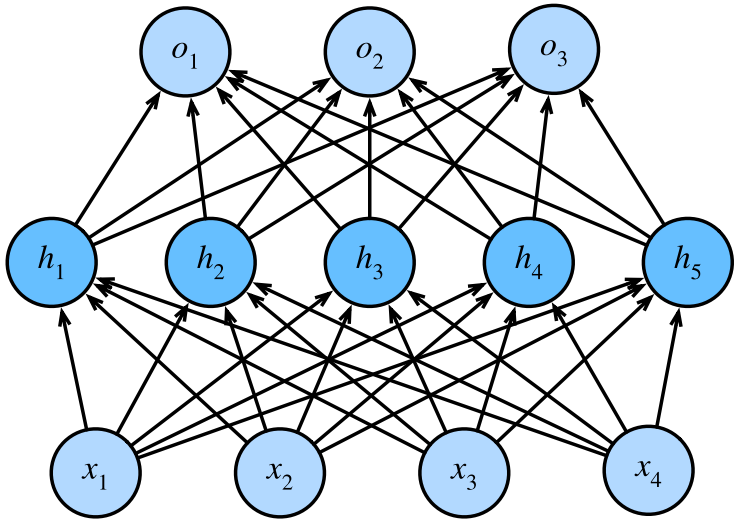


- To compensate for this fact, rule of thumb:
  - Multiply each input connection weight by keep probability (1 – p) after training.
  - Alternatively, Divide each neuron's output by keep probability during training.

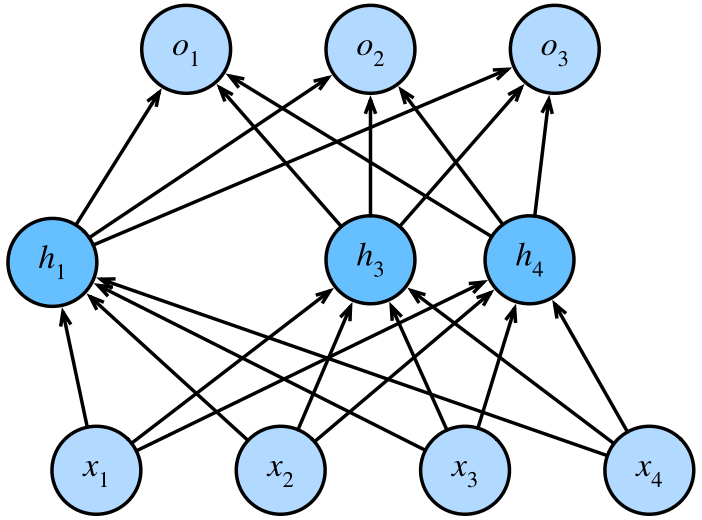
Dropout in Practice

- When we apply dropout to a hidden layer, zeroing out each hidden unit with probability p.
  - In Fig. h_2 and h_5 are removed.
  - Calculation of the outputs no longer depends on h_2 and h_5
  - Their respective gradient also vanishes when performing backpropagation.
  - Disable dropout at test time.

Before dropout



After dropout



It builds a 2-hidden-layer fully connected NN with ReLU activations and dropout to reduce overfitting.

```
class DropoutMLP(d2l.Classifier):
    def __init__(self, num_outputs, num_hiddens_1, num_hiddens_2,
                 dropout_1, dropout_2, lr):
        super().__init__()
        self.save_hyperparameters()
        self.net = tf.keras.models.Sequential([
            tf.keras.layers.Flatten(),
            tf.keras.layers.Dense(num_hiddens_1, activation=tf.nn.relu),
            tf.keras.layers.Dropout(dropout_1),
            tf.keras.layers.Dense(num_hiddens_2, activation=tf.nn.relu),
            tf.keras.layers.Dropout(dropout_2),
            tf.keras.layers.Dense(num_outputs)])
```

Code Explanation

```python
class DropoutMLP(d2l.Classifier):
    def __init__(self, num_outputs, num_hiddens_1, num_hiddens_2,
                 dropout_1, dropout_2, lr):
        super().__init__()
        self.save_hyperparameters()
        self.net = tf.keras.models.Sequential([
            tf.keras.layers.Flatten(),
            tf.keras.layers.Dense(num_hiddens_1, activation=tf.nn.relu),
            tf.keras.layers.Dropout(dropout_1),
            tf.keras.layers.Dense(num_hiddens_2, activation=tf.nn.relu),
            tf.keras.layers.Dropout(dropout_2),
            tf.keras.layers.Dense(num_outputs)])
```

https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dropout

- `class DropoutMLP(d2l.Classifier):`

  Inherits from a base classifier class (likely handling training, loss, etc.).
- `__init__(...)`

  Initializes the model with:
  - `num_outputs` : number of output units (e.g., classes)
  - `num_hiddens_1` , `num_hiddens_2` : sizes of two hidden layers
  - `dropout_1` , `dropout_2` : dropout rates for regularization
  - `lr` : learning rate
- `self.save_hyperparameters()`

  Stores the constructor arguments for later use (e.g., logging).
- `self.net = tf.keras.models.Sequential([...])`

  Defines the neural network architecture:
  1. `Flatten()` – Converts input (e.g., images) into a 1D vector.
  2. `Dense(num_hiddens_1, relu)` – First hidden layer with ReLU activation.
  3. `Dropout(dropout_1)` – Randomly drops neurons during training.
  4. `Dense(num_hiddens_2, relu)` – Second hidden layer.
  5. `Dropout(dropout_2)` – Additional regularization.
  6. `Dense(num_outputs)` – Output layer (logits, typically for classification).

Dropout

- Make sure to evaluate training loss after training.
  - If overfits training set: increase dropout rate.
  - If underfits training set: decreasing dropout rate.

- Dropout can be applied to hidden neurons
  - Between two hidden layers
  - Between last hidden layer and output layer.
- Dropout rate works as weight constraint on selected layers.

- Time-Effort trade-off: Well-tuned dropout significantly slows down convergence but results in a much better model.

Regularization: BatchNorm

BatchNorm

- Vanishing gradients problem can be alleviated with better weight initialization, better optimizers or Batch Normalization[1].
- BatchNorm most successful architectural innovations in deep learning[2].
- BatchNorm aims to stabilize distribution (over a minibatch) of inputs to a given network layer during training.

- BatchNorm Working: Operation lets model learn optimal scale and mean of each of layer's inputs.

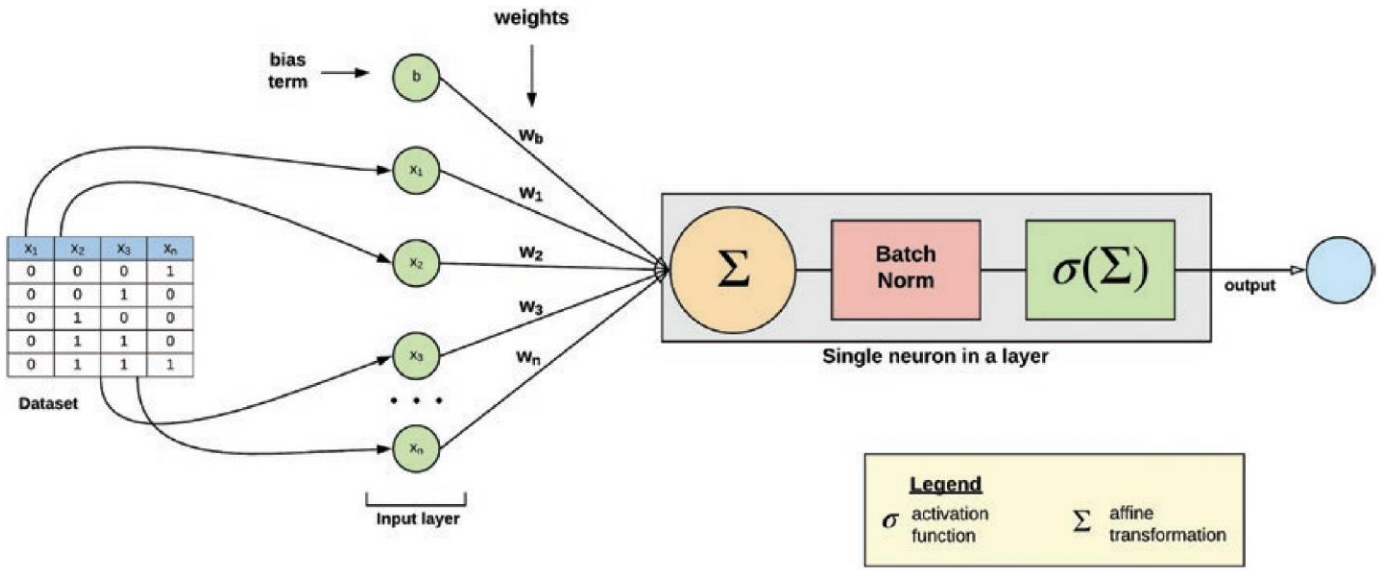
1. First add an operation in model just before or after activation function of each hidden layer. This operation simply zero-center and normalizes each input.
2. Next, scales and shifts result using scaling and shifting vectors [per layer].

1. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," Proceedings of the 32nd International Conference on Machine Learning (2015): 448–456.
2. Santurkar, Shibani, et al. "How does batch normalization help optimization?." *Advances in Neural Information Processing Systems*. 2018.

BatchNorm

- Normalizes layer activations per mini-batch, not the raw dataset globally.
- Uses batch mean and variance, which vary across batches.
- Applies learnable scale and shift after normalization.
  - Good practice is still to normalize inputs.
- BN mitigates vanishing gradients by stabilizing activation distributions, especially when used with ReLU.

For DNN:
- Speeds up convergence.
- Allows higher learning rates.
- Stabilizes training.
- Adds mild regularization.



Bisong E. (2019) More on Optimization Techniques. In: Building Machine Learning and Deep Learning Models on Google Cloud Platform. Apress, Berkeley, CA

BatchNorm

- Batch Normalization is tricky to use in RNNs but sufficient for other nets.
  - Gradient Clipping* is often used RNNs to mitigate exploding gradients problem.
    - Gradient Clipping clips the gradients during backpropagation so that they never exceed some threshold.
  - Gradient clipping does not help with vanishing gradients.
- BN acts like a regularizer reducing need for other regularization techniques (such as dropout).
- BN adds runtime penalty to neural network.
  - Training is rather slow because each epoch takes much more time when BN in use.

*Pascanu, Razvan, Tomas Mikolov, and Yoshua Bengio. "On the difficulty of training recurrent neural networks." *International conference on machine learning*. 2013.

Summary

- **Generic ML Framework**
  - Components: Data, Model, Objective Function, Optimization
  - Objective examples: MSE, Cross-Entropy, Maximum Likelihood
  - Optimization: Gradient Descent / SGD
  - Key issues: Bias, Variance, Underfitting, Overfitting
- **Artificial neuron:**
  - Weighted sum of inputs
  - Activation function
  - Bias term
- Latent (hidden) features computed via matrix transformations
- **Threshold Logic Units (TLUs)**
  - Compute weighted sum + threshold
  - Mimic biological neuron firing
  - Foundation of Perceptron

Summary

- **Perceptron**
  - Single-layer linear binary classifier
  - Uses step (Heaviside/sign) activation
  - Trained via incremental weight updates (Hebbian-inspired)
- **Dropout**
  - Randomly deactivate neurons during training (probability p)
  - Prevents co-adaptation of feature detectors
    - Trade-off:
    - Slower convergence
    - Better generalization
- **Batch Normalization (BN)**
  - Mitigates vanishing gradients
  - Often reduces need for dropout
  - Adds computational overhead